GHENT UNIVERSITY

FACULTY OF BIOSCIENCE ENGINEERING

Exercises Hydrological Modelling

Academic year: 2026 - 2027

Olivier Bonte Hans Lievens

2026-11-05

Table of contents

1 Introduction **3**

2 Practical information **4**
2.1 The repository . . . 4
2.2 Working with Quarto and Python . . . 4
2.3 Evaluation . . . 6

3 Study area and data **8**
3.1 Catchment data . . . 8
3.2 Digital terrain model . . . 8
3.3 Meteorological and discharge data . . . 9
3.4 Satellite soil moisture . . . 11

4 Exercise 1: Delineation of the Zwalm catchment **13**
4.1 Assignment . . . 13
4.2 Results and discussion . . . 15

5 Exercise 2: Implementation of the model **16**
5.1 Assignment . . . 16
5.2 Results and discussion . . . 19

6 Exercise 3: Calibration of the model **20**
6.1 Assignment . . . 20
6.2 Results and discussion . . . 21

7 Exercise 4: Uncertainty analysis of the model output **22**
7.1 Assignment . . . 22
7.2 Results and discussion . . . 23

8 Exercise 5: Data assimilation **24**
8.1 Assignment . . . 24
8.1.1 Bias-correcting the satellite soil moisture observations . . . 24
8.1.2 Data assimilation with the extended Kalman filter (EKF) . . . 24
8.2 Results and discussion . . . 25

9 Conclusion **26**

References **27**

1 Introduction

This book contains exercises for the course Hydrological Modelling at Ghent University. The focus of this set of exercises will be on rainfall-runoff modelling with a conceptual hydrological model.

The objective of the practical exercises of this course is to illustrate some of the most important steps necessary to perform a proper modelling of a medium-scale catchment in Flanders. Only some specific steps of the process will be illustrated here across 5 exercises:

1. Delineation of the catchment (Chapter 4)
2. Implementation of a hydrological model (Chapter 5)
3. Calibration of the hydrological model (Chapter 6)
4. The analysis of uncertainty in the model output of the model (Chapter 7)
5. Assimilation of observations into the model (Chapter 8).

Needless to say that for each step, there are many options, and that the choice for e.g. a specific model, calibration algorithm, or evaluation criterion depends on the specific situation, the aim of the modelling exercise, and the personal taste of the modeller. For an extensive – yet non-exhaustive – overview of different techniques in hydrological modelling, we refer to the theoretical course notes and specialised literature.

The aim of the practicals is to:

1. Make you acquainted with the advantages and disadvantages of the selected algorithms,
2. Make you aware of the potential problems related to modelling rainfall-runoff relationships, and
3. Point you to and make you reflect on existing alternatives for the algorithms selected here.

2 Practical information

Before diving into the exercises, some practical information is provided on how to effectively set up your working environment and how these exercises will be evaluated.

2.1 The repository

All code used to create this assignment is available at https://github.com/h-cel/hydrological-modelling-public. This collection of files is called a **repository**. To get started, go to the URL above, click on the green `Code` button and select `Download ZIP` (see Figure 2.1). Unzip the downloaded file and place the resulting folder at a location of your choice. This folder contains all the files you need to complete the exercises.

Note that not only the code, but also the processed data is now downloaded. You can find this data in `data/processed`. More info on the data sources and processing is given in Chapter 3.

This assignment is provided to you in two formats, up to you which one you prefer to work with:

1. A website at https://h-cel.github.io/hydrological-modelling-public/
2. A PDF document, which you can download from https://h-cel.github.io/hydrological-modelling-public/Exercises-Hydrological-Modelling.pdf

Throughout the assignment, text formatted as

> Tip
>
> *A hint*

contains tips and recommendations to help you with the exercises. If you need an extra challenge, you can find these in text blocks formatted as

> Challenge
>
> *A challenge*

These challenges are always optional and are meant for those who are interested.

2.2 Working with Quarto and Python

For general introductions, please refer to the following resources:

- Python and VS code tutorial: https://olivierbonte.github.io/imecos_software_guide/python_vs_code.html
- Quarto tutorial: https://olivierbonte.github.io/imecos_software_guide/quarto.html

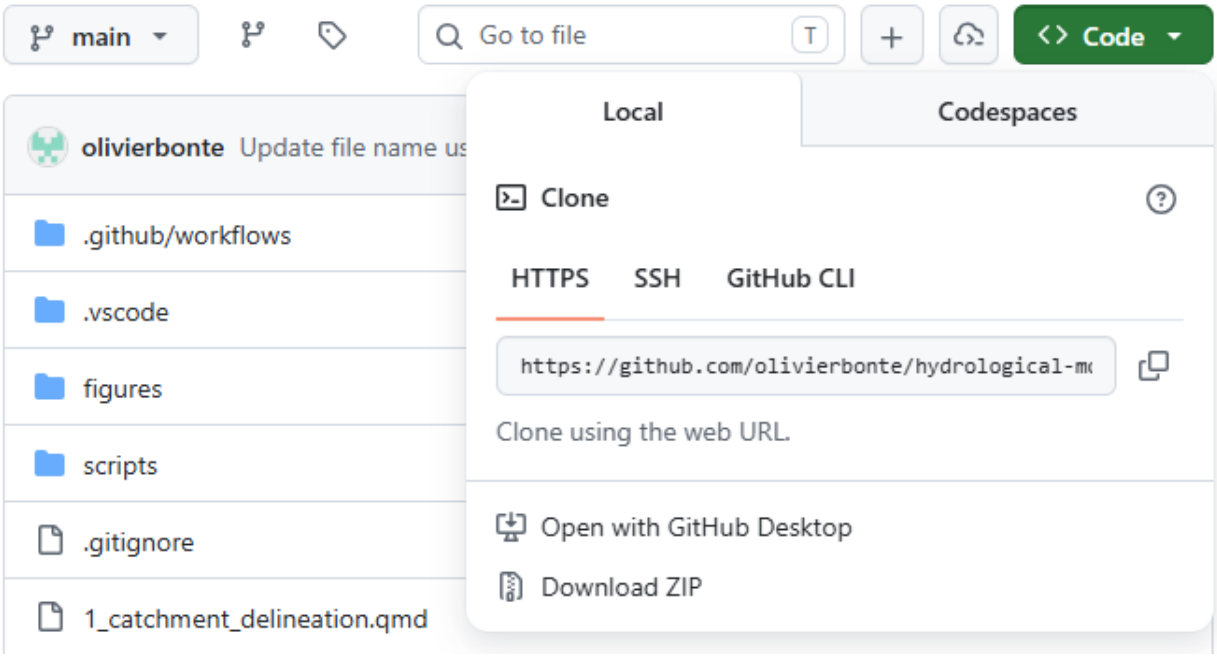


Figure 2.1: Screenshot of GitHub download button

Make sure to use the conda environment provided in `environment.yml` (`hydromod_env`)[1]. After following the guides above, you should be able to run the code in the `.qmd` files and render them into a PDF report.

The idea of this exercise set is that you will use Quarto to combine your code and your scientific report in a single document. The goal is to write a report following the principles of reproducible research[2], where others have access to all the data and code needed to obtain the same results as the one you present.

In these practicals, you will therefore **not** write a report from scratch (in Microsoft Word, Google Docs…), but instead you'll extend the provided `.qmd` files for each exercise. The files of interest are numbered from `1_...qmd` to `6_...qmd`. Here you can add Python code[3] for your figures and write your discussion in Markdown.

> **Tip 1**
>
> Don't perform all your computations inside of the `.qmd` files! Instead, make a regular `.py` file inside of the `scripts` folder for each exercise. From these `.py` files, you can save the results (e.g. metrics, output data) in (for example) a `data/solutions` folder. From this folder, you can then import the necessary results in the `.qmd` files to write your report and make your figures.
> For functions that you reuse across multiple exercises, it is recommended to add these in a `.py` file inside of the `src/hydromod` folder. This way, you can easily import these both in your `.qmd` files and in `.py` files inside the `scripts` folder. For example, in `src/hydromod/constants.py`,

[1] Only the experimental Python route for GRASS in Chapter 4, and the data download and processing scripts, need the heavier developer environment in `environment_dev.yml` (`hydromod_env_dev`)
[2] This concept is introduced in your course on Data Science in the 2nd Bachelor year of Bioscience Engineering, where you made a Quarto/Rmarkdown presentation/report for a data analysis
[3] As an example of how this is used in practice, see e.g. the source file `0_study_area_and_data.qmd` for Chapter 3

a constant `STEP_FACTOR` is defined. We can import this constant here by using:

```
from hydromod.constants import STEP_FACTOR

print(STEP_FACTOR)
```

```
0.01
```

2.3 Evaluation

The evaluation of your work will be both orally and in writing. Both forms of evaluation expect individual submissions/participation, but you are of course strongly encouraged to discuss the exercises with your colleagues during the practicals.

For the **written examination**, hand in the following by **Sunday 13/12 23:59** on Ufora:

- A PDF version of your report, which is rendered via Quarto.
- The full set of files (i.e. repository) including your solution as a `.zip` file.

By **Tuesday 10/11 23:59**, you can hand in a draft version of your results and discussion for one exercise for feedback. This will not be graded and only serves as a way to help you with writing your report and using Quarto.

Your report should be written **scientifically** and should contain at least the following:

- Per exercise, a summary of the results with reference to relevant figures and metrics and a proper discussion. Note that all tasks are related and that the results should be discussed in reference to one another in a scientific manner. The emphasis of the report should be on this part. This part should be used to draw the attention on shortcomings of the different algorithms and make a link to potential alternatives. Use the sections `Results and discussion` in `1...qmd` to `5...qmd` for writing this part down.
- A conclusion about the modelling exercise as a whole, which is written in the `6_-conclusion.qmd` file.
- Figures and tables should be labelled and referenced in the text (see Galileo for more information). Please consult the aforementioned Quarto tutorial to see how this can be achieved in `.qmd` files.
- To support your discussion, it is valuable to include references to relevant scientific papers. To add a reference, please include the relevant source in its bibtex format to `references.bib`. More information on how to handle citations in Quarto can be found here. The bibtex format is explained in this guide. By means of example, one scientific article, Moore (2007), is already included.
- Contrary to a full report/paper (see Galileo for a structure overview), you don't have to write an abstract, introduction and methods section. The concept is that these are already present in the assignment themselves. This allows you to focus on the results and discussion. Note that if you e.g. tackle one of the challenges and want to add a short extra methodology section, this is of course allowed: you can add an extra section (`## ...`) before `## Results and discussion`. ‘

As the official instruction language of the course is Dutch, you are allowed to write your report in **Dutch or English**.

> ⚠ Cross-referencing in Dutch
>
> When cross-referencing figures or tables in your report, you'll notice that by default English terms are used. E.g. `@fig-screenshot-download-button` will be rendered as Figure 2.1. This is because the language of this document for Quarto is set to English (see `lang: en` in `_quarto.yml`). The easiest way to deal with this is to use custom prefixes (see here for details). For example, using `[Figuur @fig-screenshot-download-button]` will be rendered as Figuur 2.1.

The **oral examination** will be held during the exam of this course (date TBD). It will consist of a discussion on your report and code. The focus will be on methodological choices you made, how you interpreted your results and the scientific discussion.

Lastly, note that following the guidelines on the use of Generative AI tools, you are allowed to use GenAI tools to help you with this report. Remember though that you are still responsible for the content of your report and code, and will be orally evaluated on your capacity to explain your work. Please mention at the end of your report (in the `6_conclusion.qmd` file in a new section) which GenAI tools you used (if any) and for what purpose.

3 Study area and data

For these exercises, a hydrological dataset of the Zwalm catchment will be used. The Zwalm catchment is located in the centre of Belgium and can be considered a medium-scale catchment. Its main tributary, the Zwalmbeek, is visualised in Figure 3.1. During the period 2009-2025, a dataset of daily precipitation, potential evapotranspiration and discharge is at your disposal (see Section 3.3 for more information). The average annual rainfall and potential evapotranspiration of the catchment are 783 mm and 561 mm respectively, typical values for Belgium.

As mentioned in Chapter 2, all the processed data is available at `data/processed`. If interested, you can inspect all downloading and processing steps yourself by following the instructions in `scripts/data_download/README.md` and `scripts/data_process/README.md` of the repository.

> Challenge
>
> If interested, you can run this geospatial processing code yourself. Note that however that you need to install the developer environment in `environment_dev.yml` and activate it with `conda activate hydromod_env_dev`. This can be non-trivial on Windows (see Warning 1 for details).

In the next section, more information is given on the data sources and processing steps for the different types of provided data. Note that all geographic data is provided in the Belgian Lambert 72 (BD72, EPSG:31370) coordinate reference system (CRS).

3.1 Catchment data

The data in the `data/processed/catchment_info` folder contains information on the river network and the catchment boundaries in the broad vicinity of the Zwalm catchment. The source of the river network is the *Vlaamse Hydrografische Atlas* (Vlaamse Milieumaatschappij 2026). The catchment boundaries come from the *Oppervlaktewaterlichamen en hun afstroomgebieden, 2022-2027* dataset (Vlaamse Milieumaatschappij 2023). In Chapter 4, you will be asked to delineate the catchment boundary of the Zwalmbeek yourself. The provided catchment boundaries can be used for comparison. Both files are provided as ESRI Shapefiles, called `afstroomgebied.shp` and `vlaamse_hydrografische_atlas.shp` respectively.

3.2 Digital terrain model

For the catchment delineation of Chapter 4, a digital terrain model (DTM) is provided in `data/processed/digital_terrain_model`. This DTM is derived from the *Digitaal Hoogtemodel Vlaanderen II* DTM at 1m spatial resolution (Agentschap Digitaal Vlaanderen 2014). To reduce the size of the dataset and speed up the computation, the DTM has been resampled to a spatial

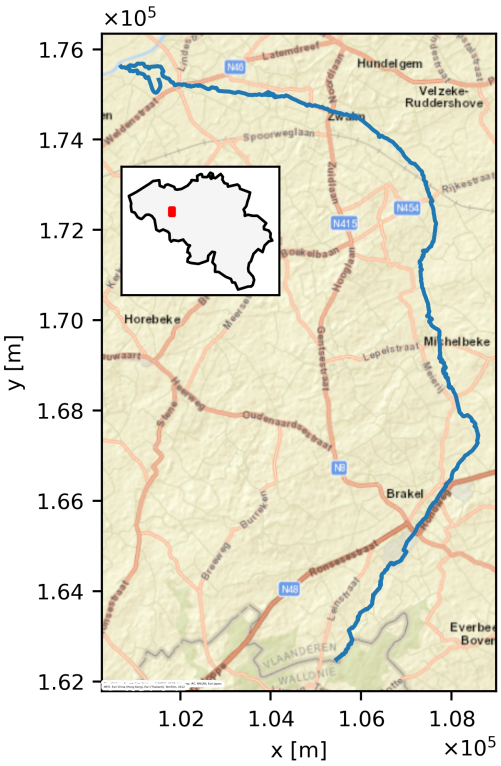


Figure 3.1: The Zwalmbeek and its location within Belgium, as denoted by the red box (CRS: EPSG:31370).

resolution of 10.0 m by averaging. The DTM is provided as a GeoTIFF file, called `DHMVII_DTM_1m_upscaled_10m.tif`.

3.3 Meteorological and discharge data

Both meteorological forcings and observed discharge are provided in the folder `data/processed/forcings_discharge`. All this data is retrieved from Waterinfo, a website which combines hydro(meteorological) data from both the Flanders Environment Agency and Flanders Hydraulics Research. Programmatic access to the data is possible via the Python package `pywaterinfo`, as demonstrated (for your information only) in `scripts/data_download/waterinfo_download.py`.

The combined forcings and discharge dataset is provided in the CSV file `forcings_discharge.csv`. An overview of the the original data sources is given in Table 3.1. Note that mm is equivalent to mm/d because of the daily time resolution. The locations of the different measurement stations are additionally shown in Figure 3.2. Some processing details:

- Precipitation (P) is the areal average of four rain gauges (Elst, Maarke-Kerkem, Zingem and Ronse) using Thiessen polygons as weighting factors[1], clipped to the Zwalm catchment as delineated by the Flemish government. When a gauge has no data for a given day, the Thiessen weights are recomputed with the remaining gauges. The Thiessen weights for each gauge when all gauges are available, are given in Table 3.1.

[1] See the Hydrology course in the Bachelor for more information on this technique

- Potential evapotranspiration (E_p) is estimated using the Penman equation at a meteorological station. Missing values are gapfilled with the daily, smoothed climatology of E_p. Smoothing occurs by applying a moving average with a centred window of 11 days.
- River discharge (Q) is measured near the mouth of the Zwalmbeek in the village of Nederzwalm. No gapfilling is applied to this variable, as it is only used for model evaluation, not as a model input.

Table 3.1: Metadata of the provided daily forcings and discharge dataset, retrieved from Waterinfo. Each variable has a corresponding measurement station, denoted by a unique station ID.

Variable	Units	Station	Station ID	Thiessen weight
Precipitation	mm	Elst OTT_P	plu06a-1066	0.74
Precipitation	mm	Maarke-Kerkem_P	P06_014	0.09
Precipitation	mm	Zingem_P	P06_040	0.17
Precipitation	mm	Ronse OTT_P	plu12a-1066	0
Potential Evapotranspiration	mm	Waregem_ME	ME05_019	
River Discharge	m^3/s	Nederzwalm/Zwalmbeek	L06_342	
Precipitation of catchment	mm	Nederzwalm/Zwalmbeek	L06_342	

```
/tmp/ipykernel_2924/4035478275.py:5: Pandas4Warning: The inplace keyword in DataFrame.reset_i
8).
See https://pandas.pydata.org/docs/dev/whatsnew/v3.1.0.html#deprecation-inplace for more deta
  df_meta.reset_index(inplace=True, names="Variable")
```

Consequently, following columns are present in `forcings_discharge.csv`:

```
['',
 'precipitation',
 'potential_evapotranspiration',
 'river_discharge',
 'precipitation_vmm_catchment']
```

Note that the first column without a name is the index column, which contains the date. The column `precipitation_vmm_catchment` is an alternative catchment precipitation product of the Flanders Environment Agency (Precipitation of catchment in Table 3.1), gapfilled with the Maarke-Kerkem gauge. It is only provided for comparison, for what follows use `precipitation`.

The correct interpretation of the date is that the data value corresponds to the daily sum (P and E_p) or daily mean (Q) of the day indicated by the date. For example, the value of P on 2020-01-01 corresponds to the total precipitation that fell during that day.

> **Tip**
>
> To read in `forcings_discharge.csv`, checkout the documentation for `pandas.read_csv()`, specifically of the `index_col` and `parse_dates` parameters, to correctly read in the dataset.

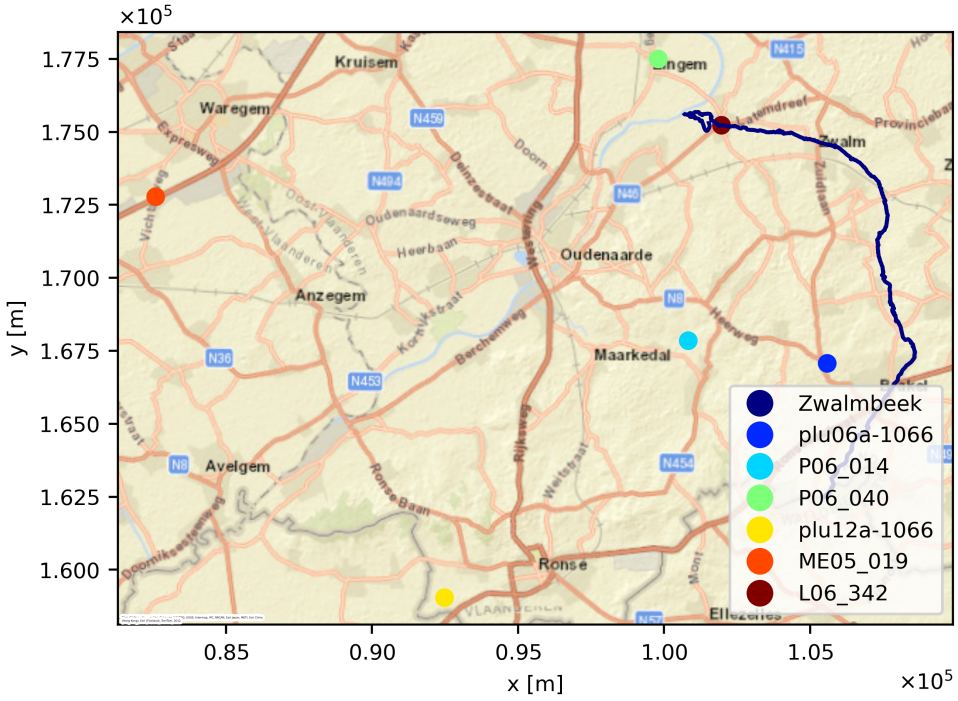


Figure 3.2: The Zwalmbeek and the measurement locations as denoted by their station ID

3.4 Satellite soil moisture

In Chapter 8, the goal is to improve the hydrological model predictions by assimilating observations of soil moisture. Here, we'll use satellite-derived surface soil moisture (SSM), which is provided in the folder `data/processed/satellite_soil_moisture`.

The original data source is the SSM product at 1 km spatial resolution over Europe from the Copernicus Land Monitoring Service (European Union's Copernicus Land Monitoring Service 2026). An example of the original data for one example date is shown in Figure 3.3. The SSM values are expressed in percent saturation and are representative of the top few centimetres (~5cm) of the soil.

For ease of use, the dataset has been processed to a timeseries by taking the average SSM value over the Zwalm catchment and subsequently removing outliers. The resulting final timeseries is called `satellite_soil_moisture.csv`, which has following columns:

```
['time', 'ssm', 'noise']
```

with `time` the date of acquisition, `ssm` the spatially averaged SSM and `noise` is the spatially averaged standard deviation of the SSM following a simple mean[2].

[2]Alternatively, one could follow the rules of propagation of uncertainty. In that case, any function f, the standard deviation is given by $\sigma_f = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 \sigma_x^2 + \left(\frac{\partial f}{\partial y}\right)^2 \sigma_y^2 + \left(\frac{\partial f}{\partial z}\right)^2 \sigma_z^2 + \cdots}$ assuming **no covariance** between $x, y, z \cdots$ and linearly approximating f. Note that an average is a linear function, so here this is not an approximation. With the average $\bar{x} = \frac{1}{N}\sum_{i=1}^{N} x_i$ as f, $\frac{\partial f}{\partial x_i} = \frac{1}{N}$ and so $\sigma_{\bar{x}}^2 = \sum_{i=1}^{N}\left(\frac{\partial \bar{x}}{\partial x_i}\right)^2 \sigma_i^2 = \frac{1}{N^2}\sum_{i=1}^{N}\sigma_i^2$. This assumption of no covariance is a strong one however, which does not necessarily hold up for soil moisture retrievals which exhibit

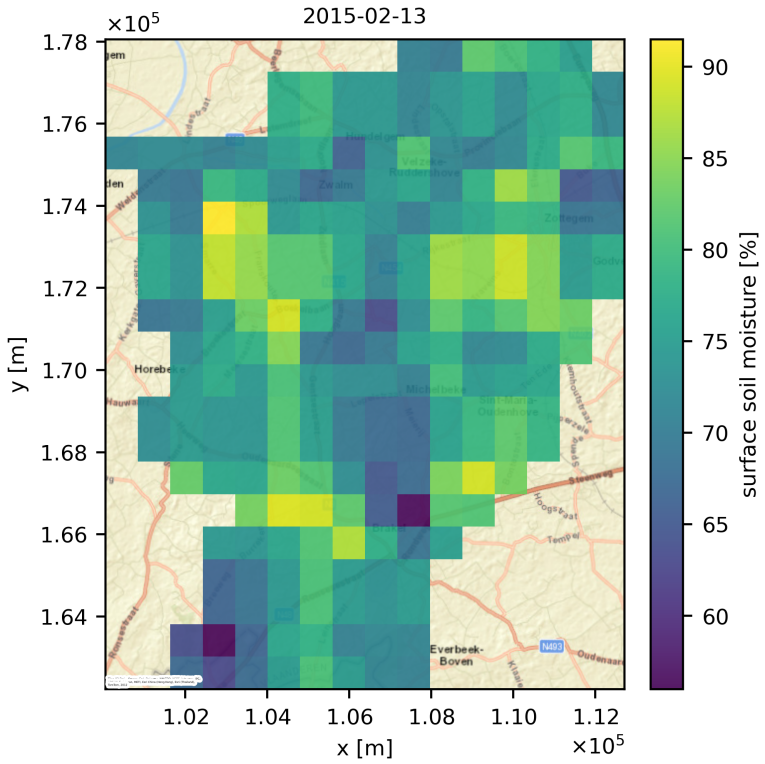


Figure 3.3: Satellite-derived surface soil moisture data at 1 km spatial resolution over the Zwalm catchment.

spatial correlation. Therefore, we make the safe assumption of perfectly correlated pixels, which increases the estimated uncertainty to $\sigma_{\bar{x}}^2 = \frac{1}{N}\sum_{i=1}^{N}\sigma_i^2$

4 Exercise 1: Delineation of the Zwalm catchment

4.1 Assignment

The delineation of a catchment based on a digital elevation model is often one of the first steps in a hydrological modelling study. The procedure is relatively straightforward, but the digital elevation model needs to be properly processed prior to the analysis. The pre-processing of the digital elevation model at least includes the removal of data gaps (i.e. spatial interpolation) and the removal of sinks (or depressions). Based on the processed DTM, the catchment can then be easily delineated using the simple principle that water always flows in the direction of the largest gradient. Different algorithms exist to do so (see course notes); here, only the gradient of every cell to its eight direct neighbours, the deterministic 8 (D8) algorithm, will be used.

> Tip 2
>
> The outlet of the Zwalm catchment is located in the vicinity of x=101673 m and y=175377 m

For this geospatial analysis, GRASS will be used, a powerful computational engine for raster, vector, and geospatial processing. It supports terrain and ecosystem modelling, hydrology, data management, and imagery processing. There are multiple ways of using GRASS. In this exercise, you can choose between 2 ways of interacting with GRASS:

- With QGIS as Graphical User Interface (GUI), all GRASS functionality can be accessed via the Processing Toolbox
- The Python GRASS interface allows programmatic access to GRASS functionality. For an in depth introduction, we refer to the documentation. In this exercise, we will use the `grass.tools` interface. The initial setup of the GRASS environment is provided in `scripts/1_catchment_delineation.py`, to which you can add your own code. All tools are accessible via the `tools` variable defined there. For example if you want to use the `r.import` function, you would call `tools.r_import(input="PATH_TO_FILE",output="OUTPUT_NAME", overwrite=True)`. Note that:
  - It is recommended to set the `overwrite=True` argument for all functions.
  - It is a general convention that the `.` defined in the GRASS function name (as found in the tool manual)is replaced by `_` in the Python function name.
  - The output of a previous function (e.g. `OUTPUT_NAME` in the example above) can be used as input for a subsequent function. For example, if you want to use the output of `r.import` as input (called `elevation`) for a slope calculation with `r.slope.aspect`, you would call `tools.r_slope_aspect(elevation="OUTPUT_NAME", slope="SLOPE", overwrite=True)`.

– In `src/hydromod/helper_functions.py`, two helper functions are defined: `get_data_array_from_raster` gives you an `xarray.DataArray` given the name (e.g. `"OUTPUT_NAME"`) of a GRASS raster layer and `plot_grass_raster` makes a plot of a GRASS raster layer (given its name as again). Use these functions to check intermediate results. Following Tip 1, you can import these functions in your script by using e.g. `from hydromod.helper_functions import plot_grass_raster`.

> ⚠ Warning 1
>
> As of 14/08/2026, installing GRASS on Windows via conda is not always stable. Therefore using GRASS from Python might not be possible. The GRASS developer team is aware of this issue, progress can be followed here. Consider using QGIS instead for now.
> The Python route is experimental and is **not** supported by the environment in `environment.yml`. It requires the developer environment: `conda env create -f environment_dev.yml` and `conda activate hydromod_env_dev`.

Irrespective of the choice of interface, an overview of all GRASS functions can be found in tool manual. Specific tips for each interface will be denoted by their logo in the steps below.

1. Start by loading the DTM (see Section 3.2). Check if it is clipped to a reasonable region of interest around the Zwalm river (bounding box of approximately $x_{\min} = 98000$ m, $x_{\max} = 116000$ m, $y_{\min} = 160000$ m, $y_{\max} = 180000$ m).
   - Use the `r.import` function to load in raster data.
2. Check if there any missing values in the DTM. If so, spatially gap-fill it:
   - Use the `r.fillnulls` function to fill in missing values. If the function fails (likely on Windows), consider choosing a different interpolation method than the default.
   - Use the `Fill nodata` tool (from GDAL).
3. Filter the digital elevation model for sinks (or depressions) and fill them. Use the `r.fill.dir` function.
4. Calculate the flow direction and accumulation into each grid cell of the domain. Use the `r.watershed` function. Make sure that D8 is used as flow direction algorithm. Furthermore, ensure that the minimum size of an exterior watershed basins is set to 500. Keep both the accumulation (i.e. how many cells drain into each cell) and the flow direction map as outputs.
5. Check if the accumulation map aligns with the actual river network of the Zwalm catchment from the *Vlaamse Hydrografische Atlas* (see Section 3.1).
   - Use `get_data_array_from_raster` to get the accumulation map as an `xarray.DataArray`. Use `geopandas.read_file` to read in the river network shapefile.
6. Find the point with the largest accumulation.
   - Rasterise the river network to the same grid as the accumulation map and extract the maximum. This point is the outlet of the catchment. Use the `make_geocube` function from the `geocube` package. Use the `like` argument.

- QGIS allows for a more visual inspection of the accumulation map. Rasterising the river network is therefore not strictly necessary and one can instead visually determine the point of largest accumulation near the Scheldt. If one would nonetheless want to rasterise the river network, use the `Rasterize (vector to raster)` tool (from GDAL).

7. Given the outlet determined in the previous step, delineate the catchment using `r.water.outlet`. Vectorise the resulting raster map of the catchment.
   - Convert the output raster to an `xarray.DataArray` using a helper function. Subsequently, the `vectorize` function from the `geocube` package can be applied.
   - Use the `Polygonize (raster to vector)` tool (from GDAL).
8. Repeat step 7 for the coordinates provided in Tip 2. Compare the resulting catchment boundary with the one obtained in step 7 and with the provided catchment boundary from the *Oppervlaktewaterlichamen en hun afstroomgebieden, 2022-2027* dataset (see Section 3.1). What are the differences between these catchment boundaries? What could be the reasons for these differences?
9. Calculate the catchment area.

In your results and discussion below, make sure to include an overview figure of the Zwalm catchment including the DTM, river network and catchment boundary.

4.2 Results and discussion

5 Exercise 2: Implementation of the model

5.1 Assignment

The lumped, conceptual rainfall-runoff model implemented here is a simplified and modified version of the FLEX model as described in Fenicia et al. (2006) and Fenicia et al. (2008). The simplifications include not considering an interception reservoir and omitting lag functions for the routing. An overview of the model structure is given in Figure 5.1. The model considers three state variables, corresponding to the volume of water in three reservoirs:

1. The unsaturated soil reservoir $S(t)$ [mm]
2. The slow reacting reservoir $S_1(t)$ [mm], conceptually representing the groundwater system
3. The fast reacting reservoir $S_2(t)$ [mm]

The model is forced with precipitation $P(t)$ and potential evaporation $E_p(t)$ (see Section 3.3 for details on the forcing data). Note that mm is used as a unit here, as it allows to implement the model independently of the catchment area[1]

Table 5.1: Model parameters together with their units, minimum and maximum value and a set of uncalibrated values

Parameter	Units	Minimum	Maximum	Value
λ	-	0.1	1.5	0.65
$S_{\max}$	mm	87	430	350
b	-	0.5	1	0.86
α	-	0.5	1	0.8
$Q_{p,\max}$	mm/d	0.75	75	34
β	-	0.05	0.1	0.06
γ	-	5	10	9.8
$S_{2,\max}$	mm	13	26	14
κ_2	mm/d	25	250	110
κ_1	1/d	0.16	0.52	0.32

By applying the conservation of mass to each reservoir, three ordinary differential equations (ODEs) can be derived to describe the change in storage in each reservoir over time.

$$\frac{dS(t)}{dt} = P_{\text{in}}(t) - E_{\text{a}}(t) - Q_{\text{p}}(t) \tag{5.1}$$

$$\frac{dS_1(t)}{dt} = P_1(t) - Q_1(t) + Q_{\text{p}}(t) \tag{5.2}$$

[1] $1\text{mm} = 1 \cdot 10^{-3}\text{m}^3/\text{m}^2$

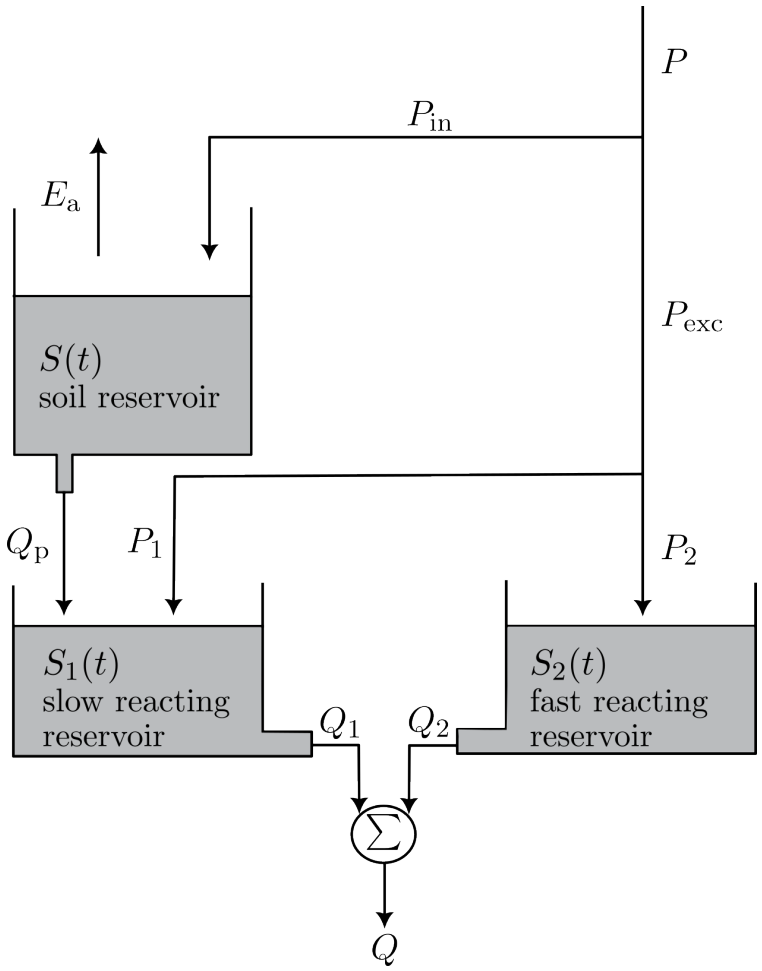


Figure 5.1: Conceptual representation of the hydrological model

$$\frac{dS_2(t)}{dt} = P_2(t) - Q_2(t) \tag{5.3}$$

In the equations above, a number of fluxes are calculated which depend on the state variables themselves. A first flux is the actual evaporation (E_a [mm/d]), which can be calculated from the potential evaporation E_p as follows:

$$E_\mathrm{a}(t) = \frac{1}{\lambda}\frac{S(t)}{S_\mathrm{max}}E_\mathrm{p}(t) \tag{5.4}$$

where λ [-] is a dimensionless model parameter and S_max [mm] is the storage capacity of the unsaturated soil reservoir $S(t)$. The infiltration in the soil reservoir P_in [mm/d] is calculated using:

$$P_\mathrm{in}(t) = \left(1 - \frac{S(t)}{S_\mathrm{max}}\right)^b P(t) \tag{5.5}$$

where b is again a dimensionless model parameter. Subsequently, the excess precipitation P_exc [mm/d] can be estimated:

$$P_\mathrm{exc}(t) = P(t) - P_\mathrm{in}(t) \tag{5.6}$$

The amount of water percolating to the groundwater storage Q_p [mm/d] is:

$$Q_\mathrm{p}(t) = Q_\mathrm{p,max}\left(1 - e^{-\beta\frac{S(t)}{S_\mathrm{max}}}\right) \tag{5.7}$$

with $Q_\mathrm{p,max}$ [mm/d] the maximum percolation volume per unit of time and β [-] a dimensionless parameter.

P_exc is again partitioned between S_2 and S_1:

$$P_\mathrm{exc}(t) = P_1(t) + P_2(t) \tag{5.8}$$

The part of the effective precipitation reaching S_2 is:

$$P_2(t) = \alpha\frac{S(t)}{S_\mathrm{max}}P_\mathrm{exc}(t)$$

with α [-] a dimensionless model parameter. P_1 is then calculated as the remaining part of the effective precipitation using Equation 5.8.

S_2 is considered as a non-linear reservoir, so that its outflow Q_2 [mm/d] can be calculated using:

$$Q_2(t) = \kappa_2\left(\frac{S_2(t)}{S_\mathrm{2,max}}\right)^\gamma \tag{5.9}$$

with $S_\mathrm{2,max}$ [mm] the storage capacity of the fast reacting reservoir, κ_2 [mm/d] the maximum outflow rate, and γ [-] a dimensionless model parameter.

S_1 on the other hand is considered as a linear reservoir, so the outflow Q_1 [mm/d] is given by:

$$Q_1(t) = \kappa_1 S_1(t) \tag{5.10}$$

where κ_1 [1/d] the reciprocal of the residence time. The total discharge Q [mm/d] is then calculated as the sum of the outflow from the fast and slow reacting reservoir:

$$Q(t) = Q_1(t) + Q_2(t) \tag{5.11}$$

Equation 5.1, Equation 5.2, and Equation 5.3 can be denoted as a general non-linear, continuous-time state-space model:

$$\frac{d\mathbf{x}}{dt} = f(\mathbf{x}, \mathbf{f}, \mathbf{p}) \tag{5.12}$$

with f a non-linear function, $\mathbf{x} = [S, S_1, S_2]^T$ the state vector, $\mathbf{f} = [P, E_p]^T$ the forcing vector, and $\mathbf{p} = [\lambda, S_{\text{max}}, b, \alpha, Q_{\text{p,max}}, \beta, \gamma, S_{2,\text{max}}, \kappa_2, \kappa_1]^T$ the parameter vector. For this exercise, the model will be implemented in discrete time using a simple forward Euler scheme[2]:

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + f(\mathbf{x}(t), \mathbf{f}(t), \mathbf{p})\Delta t$$

The time step Δt is set to 1 day, which is the same as the time step of the forcing data.

With all of the information above, implement the model based on Equation 5.1 to Equation 5.11. As a start, run it with the uncalibrated parameter values given in Table 5.1. Compare the simulated discharge with the observed discharge (see Section 3.3). Discuss the performance using relevant figures and metrics.

Tip

Some general tips and recommendations:

- Implement the model in a separate file `src/hydromod/model.py` as a function. You need to be able to reuse the model in the following exercises.
- Make sure you can run the model one time step at a time. A for-loop can then be used to run the model for the entire time period.
- Take into account the physical limitations of the state variables.
- Take into account the impact of initialisation of the state variables
- Not a single figure or metric can summarise the model performance. Nevertheless, some metrics and/or figures are (partially) linked to each other.

Challenge

The forward Euler scheme presented above is the simplest available numerical method to solve the ODEs. `SciPy` offers more advanced numerical time integration methods in `scipy.integrate.solve_ivp`. Investigate how the choice of time integration method affects the model outcome.

5.2 Results and discussion

[2]For more background info on numerical methods for ODEs, see your bachelor course on differential equations

6 Exercise 3: Calibration of the model

6.1 Assignment

As discussed in Chapter 3 of the theory notes, many different calibration algorithms exist: gradient-based, gradient-free (i.e. direct search), population-based… For this exercise, we'll focus on implementing a simple gradient-based method: steepest descent (also called gradient descent). As an objective function (ψ), the Root Mean Squared Error (RMSE) is used. The algorithm is summarised as follows:

1. The parameter vector is denoted by $\mathbf{p}^{(k)} = [p_1^{(k)}, p_2^{(k)}, \dots, p_n^{(k)}]^T$ with n the number of parameters and k the iteration number. Instead of using the raw values of the parameters, each parameter is scaled to the range $[0, 1]$ using their minimum and maximum values (see Table 5.1). For notational simplicity, these scaled parameters are still denoted by $\mathbf{p}^{(k)}$.
2. For each iteration, calculate the gradient of ψ with respect to $\mathbf{p}^{(k)}$ called $\nabla\psi^{(k)}$. The gradient is approximated numerically using the finite difference method. Since the parameters have already been scaled to $[0, 1]$ in step 1, a single, small absolute step size h can be used to perturb every parameter, rather than a parameter-specific relative perturbation.
   a. Calculate $\psi^{(k)}$, the value of ψ at $\mathbf{p}^{(k)}$.
   b. Perturb each (scaled) parameter $p_i^{(k)}$ by adding a small step size h:
   $$p_i^{(k)'} = p_i^{(k)} + h$$
   with h the square root of machine epsilon[1]. Calculate the corresponding value of the objective function $\psi_i^{(k)'}$.
   c. Approximate $\nabla\psi^{(k)}$ as
   $$\nabla\psi^{(k)} = \frac{1}{h}\left[\psi_1^{(k)'} - \psi^{(k)}, \psi_2^{(k)'} - \psi^{(k)}, \dots, \psi_n^{(k)'} - \psi^{(k)}\right]^T$$
3. Optionally (see Tip 3), normalise the gradient vector:
   $$\overline{\nabla\psi^{(k)}} = \frac{\nabla\psi^{(k)}}{||\nabla\psi^{(k)}||} = \frac{\nabla\psi^{(k)}}{\sqrt{\nabla\psi^{(k)^T}\nabla\psi^{(k)}}}$$
4. Calculate a new iteration of the parameter vector as
   $$\mathbf{p}^{(k+1)} = \mathbf{p}^{(k)} - \omega\overline{\nabla\psi^{(k)}}$$
   with ω the adaptation speed (also called step factor or step size), which is always a positive number. For simplicity, a decaying, empirical ω is used which decreases with each iteration:
   $$\omega^{(k)} = \gamma^k\omega^{(0)}$$
   with $\gamma = 0.99$ and $\omega^{(0)} = 0.01$.

[1] In Python, the machine epsilon for floating point numbers can be found using `numpy.finfo`: `np.finfo(float).eps`

The iteration steps outlined above should be repeated until convergence or a maximum number of iterations is reached.

Given the information above, now implement the steepest descent algorithm and use it to calibrate the hydrological model from Chapter 5. You can use the period 2009 - 2020 for calibration and the remainder of the dataset for validation. Evaluate the performance of the calibrated model using relevant metrics and figures. Discuss the performance of the calibrated model and the usefulness of the calibration algorithm.

Tip 3

Some general tips and recommendations:

- As steepest descent is a local optimisation algorithm, it is sensitive to local minima.
- The output of a hydrological model can be more sensitive to specific parameters.
- The choice of the objective function has a significant impact on the final calibration results.
- The criterion to stop the iterations is typically also a subjective choice, and can therefore also impact the final calibration results.
- Think about what the effect is of normalising the gradient vector. Is there an argument for not normalising it?
- As in Chapter 5, take into account the effect of the initial conditions.
- Calibration can take some computation time. Therefore, it is recommended to perform your calibration in a separate Python file `3_calibration.py`. Save the best parameter sets in e.g. a `.csv` file.

Challenge

In `SciPy`, many other calibration algorithms are available in `scipy.optimize.minimize`. Choose one of these algorithms and compare it to the steepest descent in terms of performance and computational speed.

6.2 Results and discussion

7 Exercise 4: Uncertainty analysis of the model output

7.1 Assignment

As discussed in the theory notes, uncertainty can be broadly categorised as either aleatoric or epistemic. In this exercise, we will explore the use of the **Generalised Likelihood Uncertainty Estimation (GLUE)** method to quantify the effect of the parameter uncertainty on the model output. In other words, the model structure and the forcing data are both fixed, and the uncertainty introduced due to these components is not considered here. In this case, the GLUE methodology proceeds as follows:

1. A high number (N_{tot}) of parameter sets $\mathbf{p}_{\text{i}}$ (with $i \in 1, 2, , , N_{\text{tot}}$) is generated using an effective sampling technique. If there is no a priori knowledge about the parameter values, a uniform sampling between the maximum and minimum feasible parameter value is performed. If there is some information available about the a priori distribution of the parameters, a non-uniform sampling can be performed.
2. Run a model simulation for each parameter set $\mathbf{p}_{\text{i}}$.
3. Evaluate the model performance for each parameter set $\mathbf{p}_{\text{i}}$ using a likelihood measure $L(\mathbf{p}_{\text{i}})$. In this exercise, the NSE will be used.
4. A critical likelihood threshold L_{crit} is defined to classify parameter sets as behavioural or non-behavioural. $L_{\text{crit}} = 0.4$ can be used in this exercise.
5. At each timestep, an empirical cumulative distribution function (ECDF) of the model output (i.e. discharge) is constructed using the behavioural simulations. The ECDF at timestep t is defined as:
$$P(\hat{Q}(t) \leq q) = F(q) = \frac{\sum_{i=1}^{N_{\text{beh}}} L\left(\mathbf{p}_{\text{i}} | \{\hat{Q}(t) < q\}\right)}{\sum_{i=1}^{N_{\text{beh}}} L\left(\mathbf{p}_{\text{i}}\right)}$$
with N_{beh} the number of behavioural parameter sets and $\hat{Q(t)}$ the modelled discharge at timestep t.
6. Based on the ECDF, both uncertainty bounds for the prediction interval (e.g. 95%) and the median prediction can be derived.

With the information above, implement the GLUE methodology for the conceptual rainfall-runoff model developed in Chapter 5.

> **Tip**
>
> When choosing N_{tot}, it is important to balance the computational cost against the quality of the uncertainty estimate, which is determined by N_{beh}

7.2 Results and discussion

8 Exercise 5: Data assimilation

8.1 Assignment

In Chapter 6 of the theory notes, data assimilation is introduced as a method that combines model predictions and observations to obtain an updated ("a posteriori") estimate of the model states. For this exercise, the satellite-derived surface soil moisture observations introduced in Section 3.4, denoted $y(t)$ [%], are assimilated into the calibrated hydrological model using the extended Kalman filter (EKF). Use the highest performing parameter set from Chapter 6 for this exercise.

8.1.1 Bias-correcting the satellite soil moisture observations

y is only representative of the unsaturated soil reservoir S. With $\mathbf{x}(t) = [S(t), S_1(t), S_2(t)]^T$ the state vector, the (linear) observation operator is:

$$\mathbf{H} = \begin{bmatrix} \frac{100}{S_{\max}} & 0 & 0 \end{bmatrix} \tag{8.1}$$

so that $\mathbf{Hx}(t)$ is the model equivalent of the observation. To account for the difference between $\mathbf{Hx}(t)$ (calibrated model) and $y(t)$, a bias correction is applied by rescaling using the first two statistical moments.

$$y^*(t) = \frac{\sigma_{\mathbf{Hx}}}{\sigma_y} \left(y(t) - \bar{y}\right) + \overline{\mathbf{Hx}} \tag{8.2}$$

$y^*(t)$ values that are outside of the physically meaningful range can be disregarded. Also rescale the observation uncertainty σ_y accordingly.

8.1.2 Data assimilation with the extended Kalman filter (EKF)

The state and observation vector at time step k are $\hat{\mathbf{x}}_k = [S_k, S_{1,k}, S_{2,k}]^T$ and $\mathbf{y}_k = [y_k^*]$, so that, unlike S, neither S_1 nor S_2 are directly observed. Since $\mathbf{H}$ (Equation 8.1) is already linear, only the (non-linear) rainfall-runoff model itself needs to be linearised for the EKF-application. Note that the filter only starts distinguishing an a priori from an a posteriori estimate from the second time step onwards, since the first time step is used to initialise it:

1. Initialise the state vector and error matrices.
    a. The a posteriori state estimate $\hat{\mathbf{x}}_1^+$: the initial conditions used for the calibrated model (Chapter 6).
    b. The (initial) error covariance matrix $\mathbf{P}_1^+$ starts with the values $(0.2\, S_{\max})^2$, $(0.2\, S_{2,\max})^2$ and $(0.2\, S_{2,\max})^2$ on the diagonal.

c. The process noise covariance matrix $\mathbf{Q}_k$: diagonal and constant in time at 1% of the storage capacities.
d. The observation noise covariance matrix $\mathbf{R}_k = [\sigma^2_{y^*_k}]$: time-varying, following directly from the observation uncertainty (Section 3.4).

2. Compute the a priori error covariance matrix $\mathbf{P}_k^-$, using the state transition matrix $\mathbf{A}_k$ (the Jacobian of the RHS $\mathbf{f} = [f_1, f_2, f_3]^T$ of the governing ODEs from Equation 5.12). As in Chapter 6, $\mathbf{A}_k$ is obtained via forward finite differences with step size h. With $S' = S + h$, $S'_1 = S_1 + h$ and $S'_2 = S_2 + h$:

$$\mathbf{A}_k = \begin{bmatrix} \frac{f_1(S', S_1, S_2) - f_1(S, S_1, S_2)}{h} & \frac{f_1(S, S'_1, S_2) - f_1(S, S_1, S_2)}{h} & \frac{f_1(S, S_1, S'_2) - f_1(S, S_1, S_2)}{h} \\ \frac{f_2(S', S_1, S_2) - f_2(S, S_1, S_2)}{h} & \frac{f_2(S, S'_1, S_2) - f_2(S, S_1, S_2)}{h} & \frac{f_2(S, S_1, S'_2) - f_2(S, S_1, S_2)}{h} \\ \frac{f_3(S', S_1, S_2) - f_3(S, S_1, S_2)}{h} & \frac{f_3(S, S'_1, S_2) - f_3(S, S_1, S_2)}{h} & \frac{f_3(S, S_1, S'_2) - f_3(S, S_1, S_2)}{h} \end{bmatrix}$$

.

3. The a prior error covariance matrix is computed as:

$$\mathbf{P}_k^- = \mathbf{A}_{k-1}\mathbf{P}_{k-1}^+\mathbf{A}_{k-1}^T + \mathbf{Q}_{k-1}$$

4. If an observation is available, update the state vector and error covariance, with $\mathbf{K}_k$ the Kalman gain and $(\mathbf{y}_k - \mathbf{H}\hat{\mathbf{x}}_k^-)$ the innovation:

$$\begin{cases} \mathbf{K}_k = \mathbf{P}_k^-\mathbf{H}^{\mathrm{T}}\left[\mathbf{H}\mathbf{P}_k^-\mathbf{H}^{\mathrm{T}} + \mathbf{R}_k\right]^{-1} \\ \hat{\mathbf{x}}_k^+ = \hat{\mathbf{x}}_k^- + \mathbf{K}_k\left(\mathbf{y}_k - \mathbf{H}\hat{\mathbf{x}}_k^-\right) \\ \mathbf{P}_k^+ = \left[\mathbf{I} - \mathbf{K}_k\mathbf{H}\right]\mathbf{P}_k^- \end{cases} \tag{8.3}$$

5. Otherwise, there is no update: $\hat{\mathbf{x}}_k^+ = \hat{\mathbf{x}}_k^-$ and $\mathbf{P}_k^+ = \mathbf{P}_k^-$.

6. Repeat steps 2 to 5 for each time step.

Implement the above, and use it to assimilate the satellite soil moisture observations into the calibrated hydrological model. Evaluate and discuss the impact of the assimilation on both the updated model states and the resulting discharge, using relevant metrics and figures.

> **Tip**
>
> - Inspect the Kalman gain to see how much weight is given to the observations versus the model predictions.
> - In your implementation, make sure that the calculation of fluxes given the a posteriori state is correctly handled.

8.2 Results and discussion

9 Conclusion

References

Agentschap Digitaal Vlaanderen. 2014. “Digitaal Hoogtemodel Vlaanderen II, DTM, Raster, 1 m.” Version 2014.01. Agentschap Digitaal Vlaanderen. https://metadata.vlaanderen.be/srv/api/records/f52b1a13-86bc-4b64-8256-88cc0d1a8735.

European Union’s Copernicus Land Monitoring Service. 2026. “Surface Soil Moisture 2014-Present (Raster 1 Km), Europe, Daily – Version 1.” https://doi.org/10.2909/e934b15f-7d48-4c6d-a9c6-6484488aa58f.

Fenicia, Fabrizio, Hubert H. G. Savenije, Patrick Matgen, and Laurent Pfister. 2008. “Understanding Catchment Behavior Through Stepwise Model Concept Improvement.” *Water Resources Research* 44 (1). https://doi.org/10.1029/2006WR005563.

Fenicia, F., H. H. G. Savenije, P. Matgen, and L. Pfister. 2006. “Is the Groundwater Reservoir Linear? Learning from Data in Hydrological Modelling.” *Hydrology and Earth System Sciences* 10 (1): 139–50. https://doi.org/10.5194/hess-10-139-2006.

Moore, R. J. 2007. “The PDM Rainfall-Runoff Model.” *Hydrology and Earth System Sciences* 11 (1): 483–99. https://doi.org/10.5194/hess-11-483-2007.

Vlaamse Milieumaatschappij. 2023. “Oppervlaktewaterlichamen En Hun Afstroomgebieden, 2022–2027.” Versions 2022-2027. Agentschap Digitaal Vlaanderen. https://metadata.vlaanderen.be/srv/api/records/e8f55350-5c8d-4b15-ac09-1014469c3f04.

Vlaamse Milieumaatschappij. 2026. “Vlaamse Hydrografische Atlas – Waterlopen, Toestand 03/02/2026.” Versions toestand 03/02/2026. Agentschap Digitaal Vlaanderen. https://metadata.vlaanderen.be/srv/api/records/d5a7a3df-a8a5-4516-92c4-d0128eea7fd7.